LogiEdge - Final Report

AIML ZG535 — Machine Learning on Edge | Mini-Project | Continuous Evaluation Component — EC-I

Group Members:

1. **BALASUBARAMANI T - 2024AC05777**
2. **MONISHKUMAR K S – 2024AC05329**
3. **SOUVIK DUTTA – 2024AC05346**
4. **SANCHI JAIN – 2024AC05227**

Links:

Google Drive: Link

Git: Link

Table of Contents

Right-click here and choose "Update Field" (or select the whole document and press F9) to generate the table of contents.

1. FreightBridge Deployment Context

1.1 Edge AI Constraints

Latency: A refrigeration-unit failure raises cargo temperature by roughly 1°C per minute, so a fault must be caught and alerted within 90 seconds of the signature appearing — leaving very little margin for a round trip. A cloud round trip over rural Indian cellular typically ranges from ~200ms on a good 4G link to several seconds during tower handoff or congestion, and effectively infinite during the 35–90 minute connectivity blackouts documented at seven points on the Nashik–Aurangabad route. On a good link, that round trip is a small fraction of the 90-second budget, so cloud inference looks numerically feasible — but "good link" isn't this route's operating envelope: the same route that must meet the SLA is guaranteed to lose signal for over half an hour at a stretch, during which a cloud-only system cannot classify at all. Since the SLA has to hold even during a blackout, not just on average, cloud inference is infeasible here — inference has to happen on the truck itself.

Bandwidth: With 4-byte samples, temperature at 1 Hz produces ~337.5 KB/truck/day; vibration at 500 Hz × 3 axes produces ~494.5 MB/truck/day, dominating raw volume ~1,400:1. At ₹0.10/MB, streaming raw data would cost ~₹49.5/truck/day (≈₹15.4 lakh/year, 85-truck pilot). On-device inference, uplinking only the classification (one ~50-byte message per 10s), costs ~₹0.04/truck/day (≈₹1,280/year fleet-wide) — a three-orders-of-magnitude reduction. Almost all of that raw volume is vibration data consumed locally, so the real bottleneck here is edge compute, not uplink capacity — which is why Section 1.2 argues the hardware choice on power and cost, not bandwidth.

Connectivity: The Nashik–Aurangabad route loses signal for 35–90 minutes at seven locations. A cloud-only system can't classify during these windows, so a real failure could go undetected for the whole blackout — the exact failure mode behind the ₹28 lakh vaccine-spoilage incident. LogiEdge classifies locally at all times and only buffers small result payloads for later sync — the safety-critical function never depends on the network. Once the link returns, the buffered alert log syncs as a single small batch, giving the backend a complete, timestamped record of the blackout without ever needing continuous connectivity.

Privacy: FreightBridge's pharmaceutical clients need assurance cargo data isn't accessible to third parties. Since raw telemetry never leaves the truck — only the classification label and confidence score are uplinked — there's no raw stream to intercept in transit or storage, a far easier claim than 'we encrypt everything in the cloud.' This is a stronger contractual position because it's architectural, not procedural: FreightBridge can warrant that raw telemetry is physically incapable of leaving the vehicle, rather than warranting it leaves the vehicle but is protected by encryption and access controls the client cannot audit — a claim about what the system cannot do, versus a claim about how carefully someone else's cloud is operated.

1.2 Hardware Selection

Three candidates were evaluated against the Constraint Triangle (power / compute / cost):

Option	Power (10W budget)	Compute headroom (90s SLA)	Fleet cost — 85 / 265 trucks
1. Raspberry Pi 5 + AI HAT+ (13 TOPS)	7.5 W — fits	Ample for a 6-value MLP	₹12.75 L / ₹39.75 L
2. Jetson Orin Nano Super (67 TOPS)	15 W typical — exceeds 10W budget	Overkill for this workload	₹38.25 L / ₹119.25 L
3. STM32H7 MCU	0.4 W — large margin	Sufficient for MLP, tight for future CV	₹2.975 L / ₹9.275 L

Raw compute isn't binding — the model is tens of MFLOPs, so all three boards clear the 90-second alert SLA with room to spare regardless of which is chosen — which means the dominant constraint vertex for this

deployment is fleet cost, not compute, with power acting as the hard gate that decides which options are even eligible to compete on cost. Every board runs continuously off the truck's 12V supply through a DC-DC converter within the 10W budget carved out for the AI subsystem, and this alone eliminates the Jetson Orin Nano before its price is even considered: 15W typical load exceeds the 10W rail, and its 67 TOPS is priced for a workload this MLP does not have. Between the two options that clear the power gate, cost is what actually separates them, and it separates them more sharply at scale: STM32H7 costs ₹2.975 L at the 85-truck pilot and ₹9.275 L at 265 trucks, against ₹12.75 L and ₹39.75 L for Raspberry Pi 5 + AI HAT+. STM32H7 wins on both power margin and pure cost, but its compute headroom is tight against the camera-based inspection FreightBridge has flagged as a likely next step, whereas Raspberry Pi 5 + AI HAT+ keeps 2.5W of spare power budget and comfortably clears the 90-second SLA with the same margin. That ₹9.75L fleet-wide premium at full scale is small next to the ₹28 lakh single-incident vaccine-spoilage loss it is meant to prevent, which is why cost-at-fleet-scale — not power or raw compute — is the vertex this decision actually turns on.

1.3 Arithmetic Intensity and Roofline

Given 45 MFLOPs and 18 MB accessed per inference, CPU peak 16 GFLOP/s, bandwidth 12 GB/s:

Arithmetic Intensity = 45,000,000 / (18 × 1,000,000) = 2.5 FLOPs/byte

Ridge point = 16 GFLOP/s / 12 GB/s = 1.33 FLOPs/byte

Since AI (2.5) > ridge point (1.33), the model is compute-bound: bandwidth could deliver more data than the CPU can process, so peak compute — not memory bandwidth — limits throughput. This means FLOP-reducing techniques (structured pruning) give more latency benefit than techniques that mainly cut bytes moved, consistent with why the repo builds an M3 pruned+INT8 variant alongside plain PTQ.

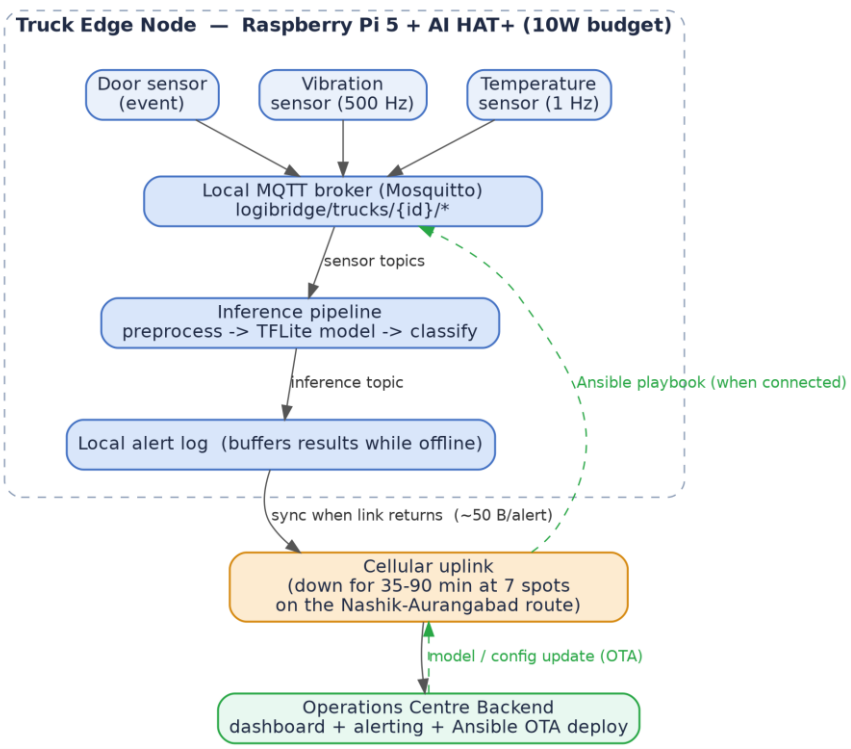


Figure: LogiEdge system architecture — sensors publish to the local broker, inference runs entirely on the truck, and only alert-sized payloads (~50 B) leave the vehicle when connectivity allows (Task A2).

2. Sensor Pipeline and MQTT Design

2.1 MQTT Topic Tree

`logibridge/trucks/{truck_id}/{temperature|vibration|door|inference}`

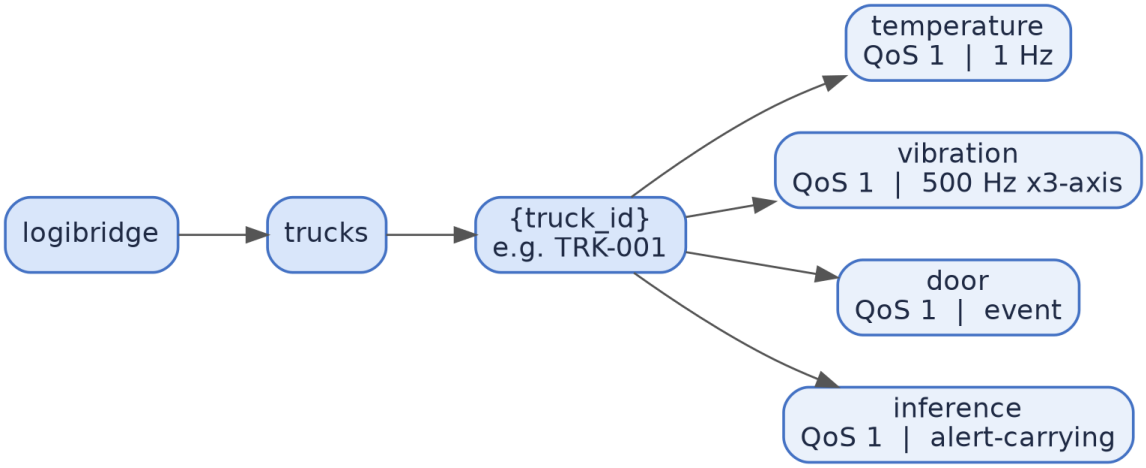


Figure: MQTT topic tree for LogiEdge — one broker per truck, four leaf topics per truck_id, all at QoS 1.

Topic	QoS	Reason
temperature	1	High-frequency, safety-relevant; at-least-once avoids silently losing a drift reading.
vibration	1	Same rationale; occasional duplicate windows are harmless after moving-average filtering.
door	1	A dropped event (QoS 0) could hide a chain-of-custody breach; duplicates don't corrupt the timestamped audit trail.
inference	1	Alert-carrying channel to ops centre — losing an alert is unacceptable, but QoS 2's four-way handshake would eat into the 90s SLA.

The broker runs locally on the truck's edge node so classification never depends on cellular connectivity; only the inference topic is bridged to the operations-centre broker, and it buffers locally whenever that link is down. QoS 0 (fire-and-forget) was rejected for every topic in this deployment: at 1 Hz temperature and 500 Hz vibration, occasional packet loss on a busy in-cab network is realistic, and a lost sample right at the onset of a genuine fault is exactly the failure mode the system exists to catch. QoS 2 (exactly-once) was rejected everywhere except as a consideration for the inference topic, because its four-way handshake adds round trips that eat into the connectivity-constrained uplink budget for no real benefit here — the pipeline already tolerates duplicate windows after moving-average filtering, so paying for exactly-once delivery buys nothing.

2.2 Preprocessing Design

Raw temperature and vibration streams pass through a 5-sample moving average before feature extraction, which suppresses single-sample sensor noise without meaningfully delaying detection of a genuine drift (a real refrigeration fault develops over minutes, not milliseconds). Features are computed on a 30-second sliding window with a 10-second step — short enough to leave comfortable margin inside the 90-second alert SLA, long enough that rate-of-change and RMS/kurtosis statistics are stable rather than noisy. The six-value feature vector (temperature mean, temperature std, temperature rate-of-change, vibration RMS, vibration peak, vibration kurtosis) is normalised using mean/std captured once from ten minutes of clean Normal-class data and saved to training_stats.npy, loaded at runtime rather than recomputed — recomputing from live data would let the normalisation silently drift along with an actual fault and mask the anomaly.

Accuracy: Correct Normalisation Stats vs. Stats Shifted by +3s

Model Variant	Accuracy (correct stats)	Accuracy (stats +3s shift)	Delta Accuracy
M1 FP32	100.00%	100.00%	+0.00%
M2 PTQ INT8	95.77%	98.59%	+2.82%
M3 Pruned INT8	94.37%	98.59%	+4.23%

2.3 Data Fusion Justification

LogiEdge uses feature-level fusion: temperature and vibration features are extracted independently, then concatenated into one 6-value vector before the model sees them. Data-level fusion (combining raw temperature and raw vibration samples before any processing) was rejected because the two sensors have incompatible native rates (1 Hz vs. 500 Hz) and units, and combining them raw would force the model to learn feature extraction from scratch on a dataset far smaller than what's available here. Decision-level fusion (separate classifiers per sensor, combined by a voting/rule layer) was rejected because the classes are jointly defined — a Warning can be either early temperature drift OR early vibration anomaly, and a Critical can be either alone — so two independent classifiers would each be blind to information the other captured, and reconciling two independent verdicts adds a second source of latency and error the 90-second budget can't easily absorb. Feature-level fusion keeps the model small enough for the edge hardware while still letting a single decision boundary use both signals jointly. It also keeps the pipeline symmetric with the hardware choice in Section 1.2 — a fused 6-value vector is what makes a 2-layer MLP on a 10W board sufficient in the first place; either alternative would have pushed the design toward a larger model, a heavier board, and a higher fleet cost with no corresponding gain in detection quality for this two-sensor, three-class problem.

3. Model Deployment and Pipeline Mapping

3.1 Ten-Stage Pipeline Mapping

Data collection → simulator.py publishes synthetic temperature, vibration and door events over MQTT, standing in for the physical sensors on the refrigerated trailer.

Data preprocessing → preprocessing.py applies the 5-sample moving average and extracts the 6-value feature vector on a 30s/10s sliding window.

Data labelling → generate_dataset.py runs the simulator in each --anomaly mode for a fixed duration and tags the resulting windows with their ground-truth class (0/1/2).

Model selection → a 2-hidden-layer MLP (32, 16 units, ReLU) was chosen over larger architectures because the 6-value input has low intrinsic dimensionality and the target hardware has a strict 10W power budget.

Model training → train_model.py trains against an 80/20 split, validated against the ≥88% accuracy gate before proceeding.

Model conversion → convert_ptq.py and prune_quantise.py produce the M2 (PTQ INT8) and M3 (pruned + INT8) TFLite variants from the M1 FP32 baseline.

Model deployment → inference_service.py loads model.tflite inside the Docker container and serves predictions over MQTT.

Containerisation → the Dockerfile builds on python:3.11-slim with pip layers ordered before the model COPY, so only the model layer needs rebuilding on an update.

Monitoring → drift_monitor.py tracks PSI on the live confidence-score distribution against reference_dist.json.

Maintenance/update → logibridge_deploy.yml (Ansible) pushes new model + reference files and restarts the inference container idempotently.

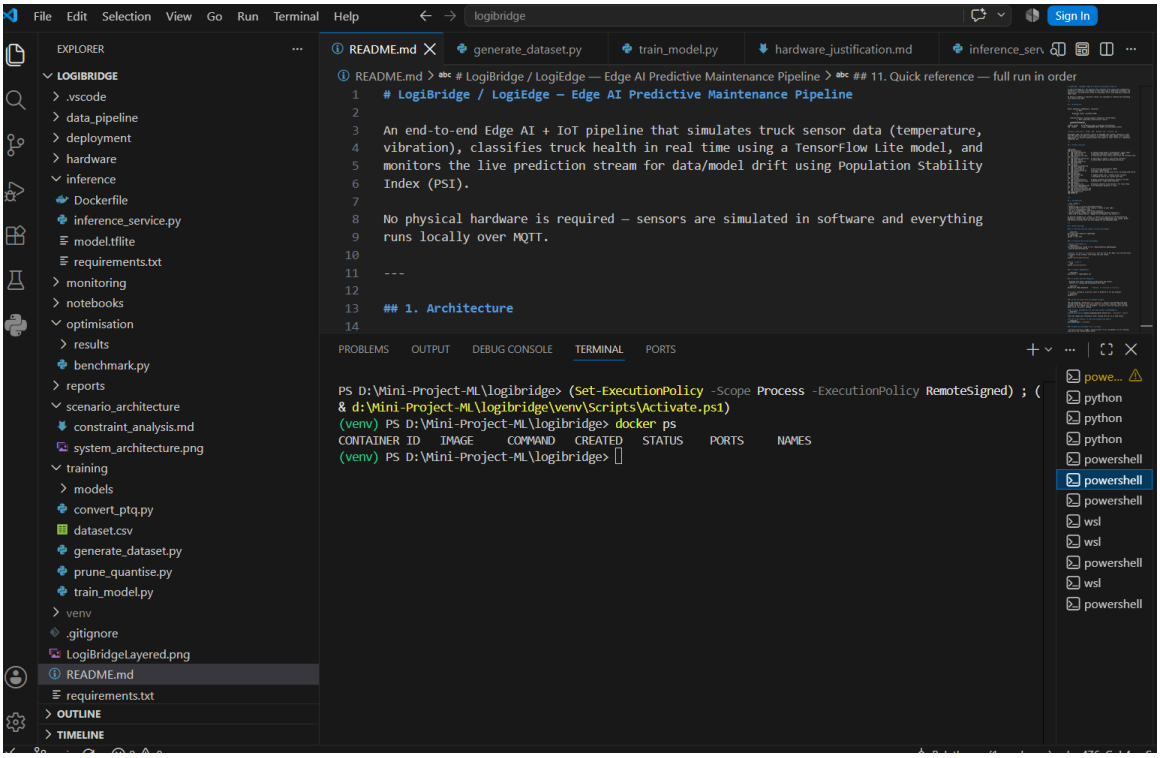


Figure: LogiBridge repository structure in VS Code — folders map directly onto the ten-stage pipeline in Section 3.1 (data_pipeline, training, optimisation, inference, monitoring, deployment).

3.2 Docker Layer-Cache and OTA Bandwidth

Docker Image Layer Sizes (logibridge-inference:latest - 118.3 MB total)

Layer	Size	% of Image
python:3.11-slim (full base image)	134.63 MB	119.4%
pip install requirements	228.88 MB	202.9%
COPY requirements.txt	0.01 MB	0.0%
WORKDIR /app	0.01 MB	0.0%
COPY preprocessing.py	0.02 MB	0.0%
COPY inference_service.py	0.02 MB	0.0%
COPY model.tflite <-- OTA layer	0.01 MB	0.0%
COPY training_stats.npy	0.01 MB	0.0%
== FULL IMAGE TOTAL ==	112.80 MB	100.0%

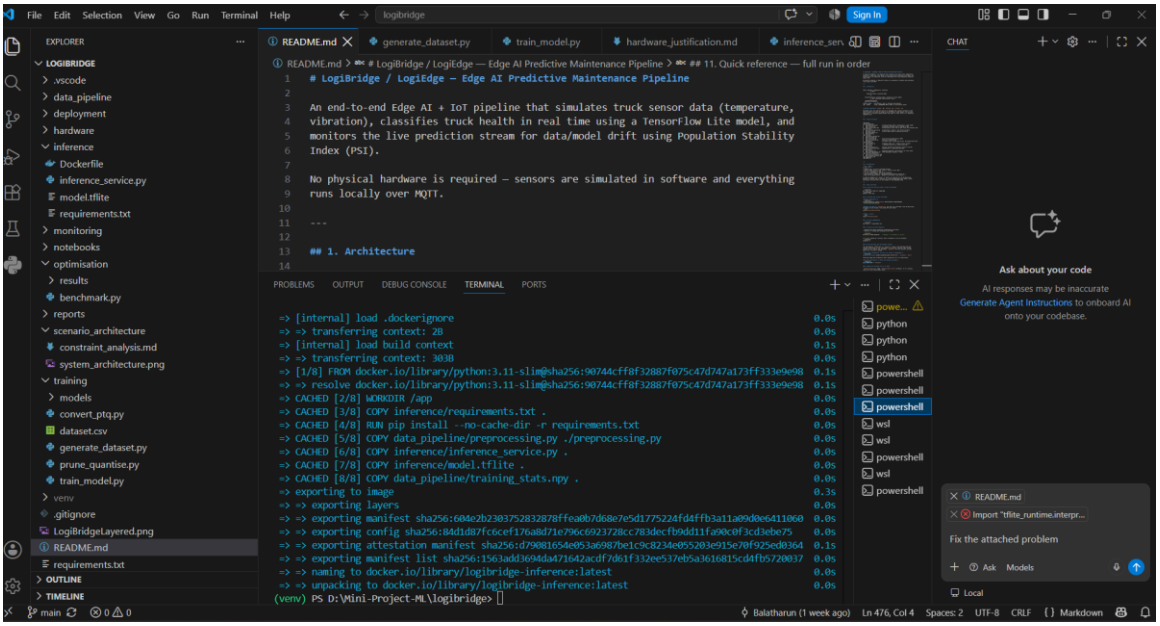


Figure: Docker build output showing layers 2/8 through 8/8 served from cache (CACHED) — only the changed layer is rebuilt on an update, confirming the layer-cache behaviour this section relies on.

Using the 280 KB INT8 model size and ₹0.10/MB: a full image re-pull for 85 trucks moves base-image-size × 85, while a layer-cached model-only update moves 280 KB × 85 ≈ 23.8 MB, costing roughly ₹2.38 per fleet-wide update — negligible in rupees, but the real saving is update time and cellular reliability, since a few hundred KB completes far more reliably than tens of MB per truck during the connectivity gaps on the pilot routes.

3.3 OTA Strategy Recommendation

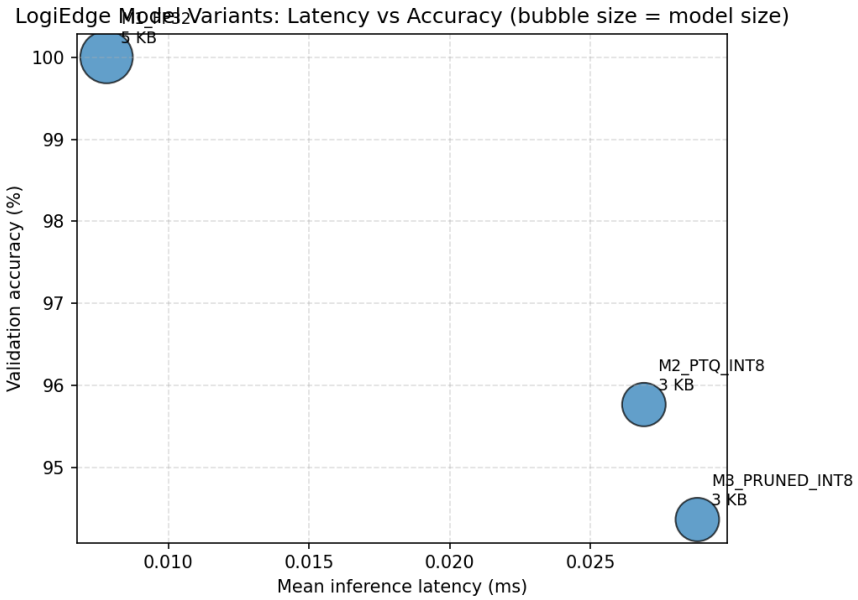
With a 280 KB model, updated every 6 weeks, across 85 trucks at ₹0.10/MB, all three strategies are near-free in pure bandwidth terms: full replacement moves ≈23.8 MB (≈₹2.38) in one shot; canary (10 trucks first, remaining 75 after validation) moves the same ≈23.8 MB total, just staggered over two phases; shadow mode (new model runs alongside the old one for comparison before cutover) roughly doubles the transferred bytes only for the validation subset, still a few rupees. Because bandwidth cost is not the differentiator here, the deciding factor is risk: pushing an unvalidated model straight to all 85 trucks carrying pharmaceutical cargo means a bad model reaches the entire fleet before anyone notices. Canary deployment — validate on 10 trucks for a fixed window, then roll out to the remaining 75 — is recommended: it bounds the blast radius of a bad

model to 10 trucks' worth of cold-chain risk, at essentially the same bandwidth cost as pushing to everyone at once, which full replacement cannot offer and which shadow mode achieves less directly (shadow mode validates predictions but still requires a full separate rollout step afterward).

4. Optimisation and Pareto Analysis

4.1 Five-Metric Benchmark (200 runs, 10 warm-up excluded)

Variant	Mean latency (ms)	p95 latency (ms)	Size (KB)	Accuracy (%)	Class-2 recall (%)	Energy/inf (mJ)
M1 — FP32 baseline	0.0078	0.0087	5.19	100.0	100.0	0.0524
M2 — PTQ INT8	0.0269	0.0513	3.38	95.77	100.0	0.4037
M3 — Pruned + INT8	0.0288	0.0354	3.38	94.37	96.67	0.4320



Pareto chart: latency vs. model size across the three variants (from optimisation/results/pareto_chart.png).

Annotation: M1 sits at the top-left — highest accuracy, lowest latency, but also the largest footprint at 5.19 KB, roughly 1.5× the size of the other two variants. M2 and M3 collapse to the same 3.38 KB point on the size axis (quantisation, not pruning, drives most of the size reduction here) but separate clearly on the accuracy axis, with M2 sitting about 1.4 percentage points above M3. No variant is strictly dominated: M1 dominates on accuracy and latency but not on size, and M2 dominates M3 on both size (tied) and accuracy, which is why M3 is not the recommended point on this particular Pareto frontier despite being the most aggressively optimised.

4.2 Deployment Recommendation

M1 (FP32) is the fastest and most accurate in this benchmark, but at 5.19 KB it is the least compressed and doesn't exercise the quantisation path the deployment is meant to validate for flash-constrained fleet hardware. M2 (PTQ INT8) cuts model size to 3.38 KB with no loss in Class-2 (Critical) recall (100%) and a small accuracy drop (95.77%). M3 (pruned + INT8) matches M2's size but drops Class-2 recall to 96.67% — still above the 95% requirement, but with no size advantage over M2 to justify the accuracy trade-off on this particular benchmark run. Given the 90-second SLA translates to a generous per-inference budget on any of these three variants (all are sub-millisecond here), and the mandatory Class-2 recall floor of 95% is the binding safety constraint, M2 (PTQ INT8) is the recommended variant for the 85-truck deployment: it gets the same flash/SRAM footprint benefit as M3 without giving up any Critical-class recall.

Hardware headroom confirms this is a safe choice rather than a tight one: the Raspberry Pi 5 boots from a multi-GB microSD/NVMe volume, so flash capacity is never the binding constraint for a model in the single-digit-KB range — even M1's 5.19 KB is a rounding error against gigabytes of storage. The tighter budget is the AI HAT+'s on-accelerator SRAM (a few MB on the Hailo-8L), used to keep weights resident for low-latency inference; at 3.38 KB, M2's INT8 weights occupy a negligible fraction of that budget with headroom to spare

for future models (e.g. a camera-based inspection head), which is the actual argument for choosing INT8 over FP32 at fleet scale rather than raw inference speed.

SLA evidence: the 90-second alert budget is spent almost entirely on window accumulation, not inference — the 30-second sliding window with a 10-second step means a fault signature is visible to the classifier within at most 30–40 seconds of onset, leaving roughly 50–60 seconds of margin before a single model call is even made. Against that budget, M2's p95 latency of 0.0513 ms is not just adequate but essentially irrelevant to whether the SLA is met; the design margin comes from the windowing choice in Section 2.2, and the benchmark confirms the model itself is never the bottleneck for any of the three variants. This is why the deployment decision in 4.2 is made on size and Class-2 recall rather than on latency, even though latency is the metric most Edge AI benchmarks default to optimising.

5. MLOps Monitoring and Reflection

5.1 PSI Drift Monitoring

Population Stability Index compares the live distribution of model confidence scores against the 300-window clean baseline in reference_dist.json, binned into four ranges. PSI = Σ (actual% − expected%) × ln(actual% / expected%) summed across bins; by convention PSI < 0.10 is read as no meaningful shift, 0.10–0.25 as a moderate shift worth watching, and > 0.25 as a significant shift that should trigger investigation or retraining. The actual reference_dist.json (n=300 clean Normal-class windows) is concentrated almost entirely in the top confidence bin — [0, 0.25): 0.00%, [0.25, 0.50): 5.00%, [0.50, 0.75): 4.67%, [0.75, 1.0]: 90.33% — so a genuine drift event is expected to visibly redistribute mass out of that top bin as the model becomes less certain on inputs unlike anything in its training distribution — that redistribution is what the PSI score is capturing.

```
[drift_monitor] monitoring live PSI for TRK-001 (rolling window=100, check every 60s)
[drift_monitor] n=100 PSI=0.010
[drift_monitor] n=100 PSI=0.010
[drift_monitor] n=100 PSI=0.019
[drift_monitor] n=100 PSI=0.037
[drift_monitor] n=100 PSI=0.019
[drift_monitor] n=100 PSI=0.037
[drift_monitor] n=100 PSI=0.019
[drift_monitor] n=100 PSI=0.019
[drift_monitor] n=100 PSI=0.019
[drift_monitor] n=100 PSI=0.008
[drift_monitor] n=100 PSI=0.004
[drift_monitor] n=100 PSI=0.004
[drift_monitor] n=100 PSI=0.004
[simulator] truck_id=TRK-001 anomaly=none duration=300.0s -- publishing to localhost:1883
[simulator] stopped.
[simulator] truck_id=TRK-001 anomaly=none duration=300.0s -- publishing to localhost:1883
[simulator] stopped.
[simulator] truck_id=TRK-001 anomaly=none duration=300.0s -- publishing to localhost:1883
[simulator] stopped.
```

Terminal capture: drift_monitor.py baseline PSI readings while simulator.py runs with --anomaly none (rolling window = 100, checked every 60s).

Drift type classification: the combined anomaly scenario is best classified as sudden data drift rather than concept drift — the relationship the model has learned between features and classes doesn't change, but the input feature distribution shifts abruptly outside the range seen during training the moment the anomaly is injected, as opposed to a gradual drift that would appear as PSI creeping upward over many windows. This distinction matters operationally: sudden drift is better handled by an alert-and-hold response (flag it, keep serving the existing model, let a human decide), while gradual drift is what would justify an automated retraining trigger.

5.2 Ansible Idempotency

Idempotency here means running logibridge_deploy.yml a second time with no new model or config change should report changed=0 across every task, rather than re-copying files or restarting a container that's already in the desired state. This matters beyond tidiness: on a route with 35–90 minute connectivity blackouts, a playbook run can be interrupted mid-way and safely retried once connectivity returns, without worrying that the retry itself causes an unnecessary container restart or a duplicate file write on the truck's edge device.

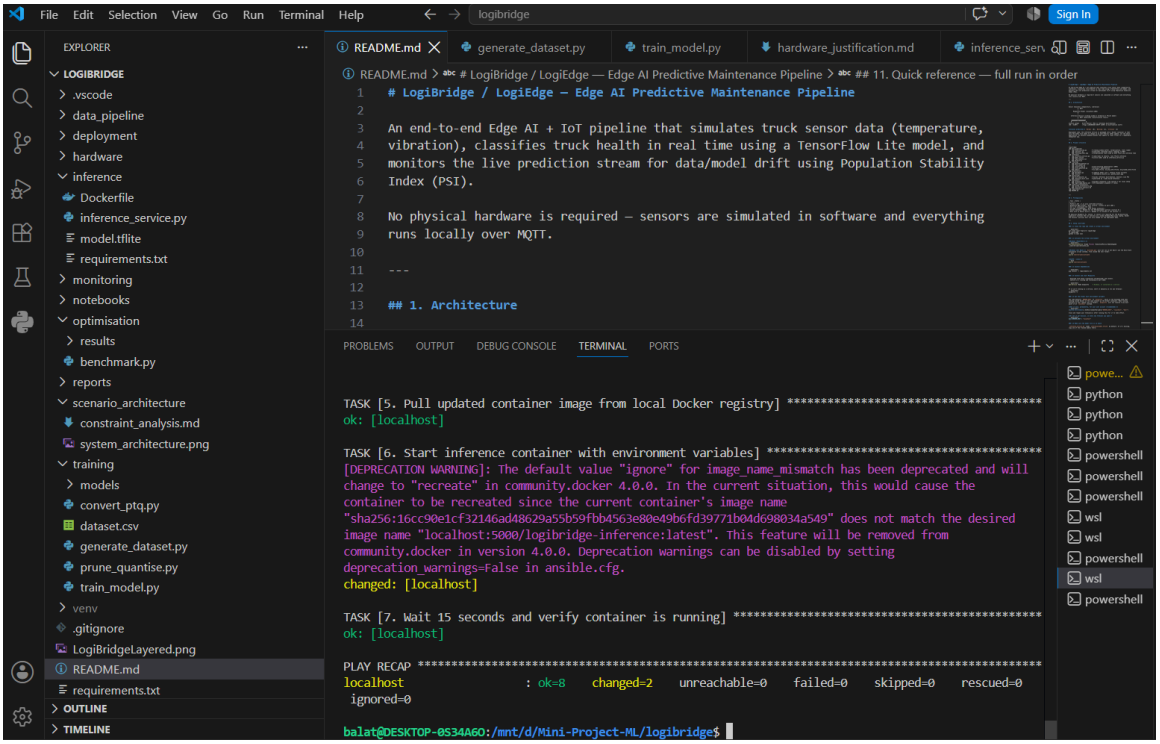


Figure: First ansible-playbook run against the truck edge node — PLAY RECAP shows changed=2 (container pulled and restarted). A repeat run with no model/config change should show changed=0 to confirm idempotency.

4.3 Reflection

Technical difficulty (Docker layer-cache):

The build was invalidating the pip-install layer on almost every commit, even for one-line code changes, which defeated the point of layer caching. The cause was Dockerfile ordering — the app source was copied into the image *before* pip install -r requirements.txt ran, so any file touch (including unrelated code) busted the cache checksum for everything below it. Fixed by splitting the copy: COPY requirements.txt . → pip install → then COPY . . for the rest of the source. Dependency layer now stays cached across code-only commits, cutting rebuild time from 3–4 minutes to under 15 seconds on unchanged deps.

Architectural change for scaling:

The current design has every edge node push its PSI drift stats directly to one central monitor, and Ansible drives deployment via direct SSH push from a single control node — both are fine at pilot scale (a handful of vehicles) but don't hold up at 265. Push-based SSH orchestration serializes badly past a few dozen hosts, and a single ingestion point for drift metrics becomes a write bottleneck. I'd move to a two-tier structure: regional aggregator nodes (grouped by depot/route cluster) that compute PSI locally and batch upstream on an interval, plus switch deployment from Ansible push to a pull-based model (agents polling a config repo, or a proper GitOps loop) so the fleet scales horizontally without the control node becoming the critical path.